Modern Cryptography and AI/ML Security

LECTURE XX

Privacy-Preserving Federated Learning

IISER THIRUVANANTHAPURAM

Course ID: DSC 4110/6111 **Instructor:** Dr. Laltu Sardar **Date:** October 29, 2026

1 Federated Learning: Motivation and System Model

1.1 Motivation

Consider n parties, where party P_i holds a private dataset D_i and trains a local model M_i. The parties want to learn a single global model without revealing their raw datasets.

- **Collaborative learning:** obtain a global model M_G using knowledge from multiple parties.
- **Data privacy:** keep each dataset D_i local and exchange model updates instead of raw data.

1.2 System Model

Let P_i denote party i, D_i its local dataset, M_i its local model, and f_i the accuracy of that model. Let $\widehat{f}$ denote the accuracy of the global model.

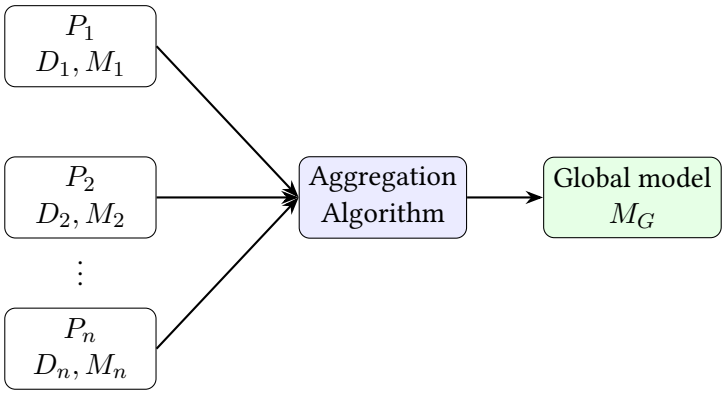


Figure 1: Centralized federated learning: parties share model updates, not their raw datasets.

1.3 System Architectures

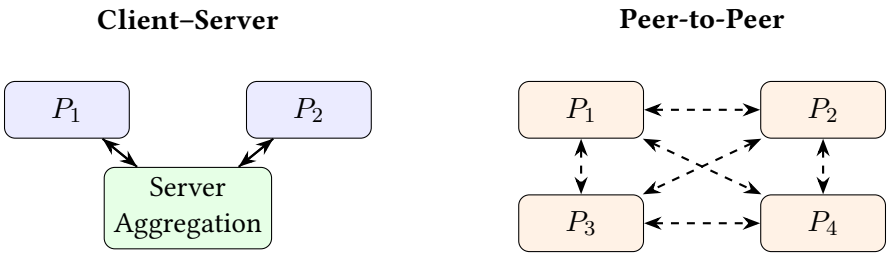


Figure 2: Centralized aggregation versus decentralized model exchange.

- **Client–server:** the server coordinates training, aggregates updates, and distributes the global model.

- **Peer-to-peer (P2P):** parties exchange model information directly, without a central aggregator.

In P2P learning, communication may follow a cyclic or random transfer strategy. Removing the central server changes the coordination model, but does not automatically ensure privacy.

1.4 Validity and Accuracy

A global model is useful only if combining knowledge from multiple parties improves learning or produces performance close to that of centralized training. Since the global model is trained without collecting raw datasets at a central location, we need criteria to evaluate its validity and accuracy.

Valid Global Model

Let $P_1, \ldots, P_n$ be n parties, and let f_i denote the accuracy of the local model M_i of party P_i. Let $\widehat{f}$ denote the accuracy of the global model M_G, measured using the same evaluation metric. The global model is called valid if

$$\widehat{f} > f_i \quad \text{for at least one } i \in \{1, \ldots, n\}.$$

δ-Accurate Global Model

Let $D = \bigcup_{i=1}^{n} D_i$ be the union of the parties' datasets, and let $M^\star$ be a model obtained by centrally training on D. Denote the accuracy of $M^\star$ by $f^\star$ and the accuracy of the federated global model by $\widehat{f}$, measured on the same evaluation distribution. For $\delta > 0$, the global model is called δ-accurate with respect to centralized training if

$$\left|\widehat{f} - f^\star\right| < \delta.$$

Interpretation. Validity requires the global model to outperform at least one local model, whereas δ-accuracy requires its accuracy to be close to that of the centralized reference model. The latter measures the performance gap introduced by federated training. Neither property alone guarantees privacy; privacy must be addressed separately.

1.5 Generic Federated Learning Algorithm

Federated learning enables multiple parties to collaboratively train a global model without sharing their raw datasets. The server coordinates training by distributing the current global model to selected clients, aggregating their locally trained parameters, and updating the global model over multiple communication rounds.

Algorithm Overview.

1. **Initialization:** The server initializes the global model parameters $\theta^{(0)}$.
2. **Client Selection and Distribution:** In each round t, the server selects a set of clients S_t and distributes the current global model $\theta^{(t)}$ to them.
3. **Local Training:** Each selected client i initializes its local model with $\theta^{(t)}$ and trains it on its private dataset D_i, obtaining updated parameters $\theta_i^{(t+1)}$.
4. **Aggregation:** The server aggregates the received local models. For example, FedAvg computes the next global model as
$$\theta^{(t+1)} = \sum_{i \in S_t} \frac{|D_i|}{\sum_{j \in S_t} |D_j|} \theta_i^{(t+1)}.$$
5. **Iteration:** The server repeats client selection, distribution, local training, and aggregation until the stopping criterion is satisfied. It then outputs the final global model.

Main Concerns. The two main concerns in federated learning are:

1. **Aggregation Algorithm:** How should the locally trained models from different parties be combined to obtain an effective global model?
2. **Global and Local Model Integration:** How should the current global model be incorporated into local training so that each party benefits from the knowledge learned by other parties while adapting the model to its own private dataset?

1.6 Model Aggregation Algorithms

In each training round, participating parties obtain local model parameters by training on their respective datasets. These models may differ because the parties have different data distributions, dataset sizes, and local training outcomes. An aggregation algorithm is therefore needed to combine local models into a global model that can be redistributed for the next training round.

1.6.1 Federated Averaging (FedAvg)

FedAvg is a widely used aggregation algorithm that computes a dataset-size-weighted average of local model parameters.

Let S_t denote the set of participating parties in round t, $N_i = |D_i|$ the size of party i's dataset, and $\theta_i^{(t)}$ its locally trained parameters. The server computes

$$\boxed{\theta^{(t+1)} = \sum_{i \in S_t} \frac{N_i}{\sum_{j \in S_t} N_j} \theta_i^{(t)}}$$

The weight assigned to party i is proportional to its dataset size. Consequently, parties with larger datasets contribute more to the aggregate. FedAvg is computationally simple, but parameter averaging does not guarantee improved accuracy for every data distribution. Moreover, FedAvg alone does not provide a formal privacy guarantee.

1.6.2 Uniform Averaging

Unlike FedAvg, which assigns weights proportional to local dataset sizes, uniform averaging assigns equal weight to every participating party. It is suitable when each party should have equal influence on the global model, regardless of its dataset size.

Let S_t be the set of participating parties in round t, and let $\theta_i^{(t)}$ denote the locally trained parameters of party i. The global model is computed as

$$\boxed{\theta^{(t+1)} = \frac{1}{|S_t|} \sum_{i \in S_t} \theta_i^{(t)}}$$

Comparison with FedAvg. FedAvg weights each local model according to its dataset size, whereas uniform averaging treats all participating parties equally. Consequently, uniform averaging may give smaller datasets greater relative influence, while FedAvg gives larger datasets greater influence. Neither method is guaranteed to perform better in all settings, particularly when local data distributions differ.

1.6.3 Other Aggregation Algorithms

FedAvg and UniformAvg are primarily aggregation and training algorithms. There are many algorithms like, FedProx, SCAFFOLD, FedNova, etc. that modify the optimization procedure, while robust aggregation methods focus on how updates are combined. These algorithms do not automatically guarantee privacy; additional protection may be required.

1.7 Global and Local Model Integration

In federated learning, global and local models are integrated through iterative local training. The server distributes the current global model parameters $\theta^{(t)}$ to selected clients. Each client initializes its local model using these parameters and trains it on its private dataset D_i. Thus, the global model provides a common starting point, while local training adapts the parameters to the client's data.

Formally, the local training objective of client i is

$$F_i(\theta) = \frac{1}{|D_i|} \sum_{(x,y)\in D_i} \ell(\theta; x, y),$$

where ℓ is the loss function. Starting from $\theta_{i,0}^{(t)} = \theta^{(t)}$, the client updates its parameters using local optimization:

$$\theta_{i,k+1}^{(t)} = \theta_{i,k}^{(t)} - \eta \nabla F_i(\theta_{i,k}^{(t)}),$$

where η is the learning rate and k denotes the local training step. After local training, the client sends its updated parameters to the server, which aggregates the received models to obtain $\theta^{(t+1)}$.

Importantly, the global and local models are not simply concatenated or added together. Instead, the global parameters initialize local training, and the locally updated parameters contribute to the next global model through aggregation. This process allows knowledge learned across participating clients to influence subsequent rounds of local training without requiring the exchange of raw data.

2 Privacy-Preserving Federated Learning

Although federated learning keeps raw datasets local, model updates such as gradients and weights, as well as the final model, may still leak sensitive information. A curious server or malicious participant may analyze these outputs to infer private information about individual training records.

Privacy-preserving federated learning therefore combines collaborative training with appropriate privacy mechanisms selected according to the threat model, privacy requirements, and acceptable utility loss.

2.1 Privacy-Preserving Techniques

The survey groups privacy-preserving FL (PPFL) techniques into four categories.

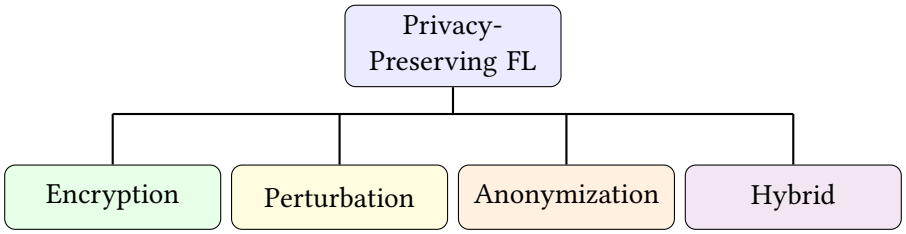


Figure 3: Main categories of privacy-preserving techniques in federated learning.

- **Encryption-based:** homomorphic encryption, secret sharing, and secure multiparty computation (MPC). These protect information during computation or aggregation.
- **Perturbation-based:** differential privacy (DP) and other noise-based methods. These reduce the information revealed by data or model outputs.
- **Anonymization-based:** methods such as k-anonymity and l-diversity, primarily designed for structured data.
- **Hybrid:** combines multiple protection mechanisms to address different leakage risks.

In this lecture note, we focus on Encryption-Based methods.

2.2 Adversarial Threat Model

Before designing a privacy-preserving federated learning (PPFL) scheme, we must first define the adversary and specify its type, capabilities, and power. The choice of privacy mechanisms depends on the adversary we aim to protect against and the information we want to keep private.

To design an appropriate PPFL scheme, we must first answer several questions about the adversary, its capabilities, potential attacks, and the information that must be protected. The five-W framework provides a minimum set of questions to organize this analysis; additional questions may be necessary depending on the application and the security requirements.

Question	Focus	Examples
Who?	Adversary type	Internal participants, server, external users, eavesdroppers
What?	Adversary behavior	Passive observation or active manipulation
When?	Attack phase	Training or inference
Where?	Exposed object	Weights, gradients, or final model
Why?	Inference objective	Recovering training data or inferring sensitive attributes and membership

By answering these questions, we establish the threat model and identify the privacy requirements. We can then design a PPFL scheme accordingly, selecting suitable mechanisms to address the specified adversary and its capabilities while balan cing privacy, model utility, and computational overhead.

3 Preliminaries

3.1 Homomorphic Encryption

Homomorphic encryption (HE) is a public-key encryption technique that enables computations to be performed directly on ciphertexts. Let $\mathcal{M}$ be the plaintext space and $\mathcal{C}$ the ciphertext space. A homomorphic encryption (HE) scheme consists of four algorithms $\mathsf{HE} = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Eval})$:

- $\mathsf{KeyGen}(1^\lambda) \to (\mathsf{pk}, \mathsf{sk})$: On input the security parameter λ, generates a public key pk and a secret key sk.
- $\mathsf{Enc}(\mathsf{pk}, m) \to c$: On input the public key and plaintext $m \in \mathcal{M}$, outputs a ciphertext c.
- $\mathsf{Dec}(\mathsf{sk}, c) \to m$: On input the secret key and ciphertext c, recovers the plaintext m.
- $\mathsf{Eval}(\mathsf{pk}, f, c_1, \ldots, c_n) \to c_f$: On input a function f and ciphertexts $c_i = \mathsf{Enc}(\mathsf{pk}, m_i)$, outputs a ciphertext c_f such that, for supported functions f,

$$\mathsf{Dec}(\mathsf{sk}, c_f) = f(m_1, \ldots, m_n).$$

Here, $\mathcal{M}$ denotes the plaintext space, and c_f encrypts the result of evaluating f on the underlying plaintexts. In fully homomorphic encryption (FHE), the evaluation algorithm supports both addition and multiplication on ciphertexts:

$$c_{\mathsf{add}} = \mathsf{Eval}(\mathsf{pk}, +, c_1, c_2),$$
$$c_{\mathsf{mult}} = \mathsf{Eval}(\mathsf{pk}, \times, c_1, c_2),$$

such that

$$\mathsf{Dec}(\mathsf{sk}, c_{\mathsf{add}}) = m_1 + m_2,$$
$$\mathsf{Dec}(\mathsf{sk}, c_{\mathsf{mult}}) = m_1 m_2,$$

where $c_i = \mathsf{Enc}(\mathsf{pk}, m_i)$ for $i \in \{1, 2\}$.

A *fully homomorphic encryption* (FHE) scheme supports arbitrary circuits, subject to its correctness and computational constraints. Homomorphic operations are defined over the scheme's plaintext domain, which may be a finite ring. Additively and multiplicatively homomorphic schemes support ciphertext addition and multiplication, respectively, while fully homomorphic encryption (FHE) supports arbitrary circuits. For example, BFV supports both addition and multiplication over encoded plaintexts. In federated

learning, clients encrypt their local gradients, allowing the server to aggregate them without decrypting individual gradients. With multiparty HE, decryption can be distributed among participants so that no single participant holds the complete secret key.

4 Secure Aggregation Using Multiparty Homomorphic Encryption

Hosseini et al. [2] propose a Robust Secure Aggregation (RSA) protocol based on multiparty homomorphic encryption (MPHE), Shamir's secret sharing, and the BFV encryption scheme. The objective is to allow a central server to aggregate clients' gradients without accessing their individual plaintext gradients, while tolerating client dropouts.

Let $\mathcal{C} = \{C_1, \ldots, C_N\}$ be the set of clients, $S_t \subseteq \mathcal{C}$ the participating clients in round t, and k the minimum number of clients required for collaborative decryption.

1. Setup and Distributed Key Generation. Each client C_i independently samples a secret-key contribution s_i. It generates Shamir secret shares of s_i using a degree-$(k-1)$ polynomial and securely distributes the shares among the clients. Each client combines the shares received from other clients to obtain its share s_i' of the collective secret

$$s = \sum_{i=1}^{N} s_i.$$

The clients also contribute to constructing a common BFV public key cpk. The server aggregates the public-key contributions to obtain cpk, which is made available to the clients. No single participant is given the complete collective secret key.

2. Local Training and Encryption. At round t, the server distributes the current global model $\theta^{(t)}$ to the participating clients. Each client trains locally on its private dataset D_i and computes its gradient $g_i^{(t)}$. It encrypts this gradient using the collective public key:

$$ct_i^{(t)} = \mathsf{Enc}_{cpk}\big(g_i^{(t)}\big).$$

Each client sends $ct_i^{(t)}$ to the server instead of sending its plaintext gradient.

3. Homomorphic Aggregation. Using the additive homomorphism of BFV, the server computes

$$ct^{(t)} = \sum_{i \in S_t} ct_i^{(t)} = \mathsf{Enc}_{cpk}\left(\sum_{i \in S_t} g_i^{(t)}\right),$$

where the equality is understood with the encoding and arithmetic of the BFV scheme. The server can therefore aggregate the encrypted gradients without decrypting them individually. It broadcasts the aggregate ciphertext to the clients.

4. Threshold Collaborative Decryption. The server identifies at least k available clients and provides the required reconstruction coefficients. Each selected client uses its secret-key share to compute a partial decryption value for the aggregate ciphertext. To protect information about its secret share, the client adds a random smudging-noise term before sending the partial value to the server.

The server combines the partial decryption values to recover the aggregate gradient:

$$G^{(t)} = \sum_{i \in S_t} g_i^{(t)}.$$

Thus, decryption is performed collaboratively, without requiring any one client to possess the complete collective secret key. The threshold mechanism permits decryption when some clients drop out, provided that at least k suitable clients remain available.

5. Global Model Update. After recovering the aggregate gradient, the server updates the global model. For example, using the average gradient,

$$\theta^{(t+1)} = \theta^{(t)} - \eta_t \frac{G^{(t)}}{|S_t|},$$

where η_t is the learning rate. The server distributes the updated model, and the clients repeat local training in the next round.

Security Discussion. The individual gradients remain encrypted during transmission and server-side aggregation. The collective secret key is distributed among clients, while smudging noise protects the partial decryption values. The paper analyzes security under a passive semi-honest adversarial model and establishes conditions on cryptographic and noise parameters for correctness and security. These guarantees do not automatically extend to arbitrary malicious behavior or unrestricted collusion.

Scope of the Scheme. The protocol's primary contribution is secure aggregation of client gradients, rather than complete protection of every aspect of federated learning. It uses a central server and clients, with no separate model owner holding the complete decryption key. Introducing a distinct model owner would require an extension of the original protocol.

4.1 Privacy-Preserving Federated Learning in a Peer-to-Peer Setting

In peer-to-peer (P2P) federated learning, clients collaboratively train a model without a central aggregation server. Each client C_i maintains a private dataset D_i and a local model parameterized by θ_i. The participating clients follow a predefined communication schedule, such as cyclic or random selection. At each round, the current model holder selects the next participant according to this schedule and forwards the current model to it. The receiving participant trains the model using its local dataset, updates the parameters, and forwards the updated model to the next participant. This process continues until a predefined stopping condition is satisfied.

Algorithm: Serverless P2P Federated Learning. Let $\mathcal{C} = \{C_1, \ldots, C_N\}$ be the set of clients, and let $\theta^{(0)}$ be the initial model. The next-client selection function $\mathsf{Next}(\cdot)$ determines the communication order.

Input: Clients $\mathcal{C}$, local datasets D_i, initial model $\theta^{(0)}$, number of rounds T.

Output: Final model $\theta^{(T)}$.

1. Initialize the current model holder as C_1.
2. For each round $t = 0, \ldots, T-1$:
   (a) The current holder selects the next client $C_j = \mathsf{Next}(C_i)$ according to the predefined cyclic or random selection rule.
   (b) The current holder sends $\theta^{(t)}$ to C_j.
   (c) Client C_j trains the received model on its local dataset D_j:
   $$\theta^{(t+1)} = \mathsf{Train}(\theta^{(t)}, D_j).$$
   (d) Client C_j becomes the current model holder and forwards the updated model in the next step.
3. Return the final model after the stopping condition is satisfied.

In this serverless setting, model updates are propagated directly between clients, eliminating the need for a central aggregator. However, this sequential procedure differs from conventional federated averaging, in which multiple clients train in parallel and their updates are aggregated.

4.2 Peer-to-Peer Federated Learning with a Rotating Aggregator

In a serverless peer-to-peer (P2P) federated learning setting, one participating client can temporarily act as an aggregator, performing the role of a cloud server without requiring a dedicated central server. At each round, the current aggregator receives model updates from participating clients, aggregates them into a single model, and forwards the updated model to the next selected participant, which becomes the aggregator in the subsequent round. The selection can follow a predefined cyclic order or a randomized schedule.

Let $\mathcal{C} = \{C_1, \ldots, C_N\}$ be the set of clients, and let $\theta^{(t)}$ denote the global model at round t. The selected clients locally train the received model on their respective private datasets D_i and produce updated parameters $\theta_i^{(t+1)}$. The current aggregator C_a computes

$$\theta^{(t+1)} = \sum_{i \in S_t} w_i \theta_i^{(t+1)}, \qquad \sum_{i \in S_t} w_i = 1,$$

where S_t is the set of participating clients and w_i are aggregation weights, for example, $w_i = |D_i| / \sum_{j \in S_t} |D_j|$ for FedAvg. The aggregator then forwards $\theta^{(t+1)}$ to the next selected participant, and the process repeats until the stopping criterion is met.

Problem. Although this approach eliminates the need for a dedicated cloud server, privacy challenges remain:

- If all participants share the same decryption key, the aggregator can decrypt individual encrypted updates and access each client's local model.
- The temporarily selected aggregator can access the local model updates received from participating clients.
- Applying homomorphic encryption (HE) requires compatible encryption keys among participants, but the aggregator must obtain the aggregated model in plaintext to forward it for subsequent local training.
- Therefore, conventional HE with a shared decryption key does not adequately protect individual model updates against an aggregator that is itself a participating client.

References

[1] X. Yin, Y. Zhu, and J. Hu, "A Comprehensive Survey of Privacy-preserving Federated Learning: A Taxonomy, Review, and Future Directions," *ACM Computing Surveys*, vol. 54, no. 6, Article 131, pp. 1–36, Jul. 2021. doi: 10.1145/3460427.

[2] E. Hosseini, S. Chen, and A. Khisti, "Secure Aggregation in Federated Learning using Multiparty Homomorphic Encryption," *arXiv preprint arXiv:2503.00581*, 2025.